

제10장: 비전 트랜스포머(Vision Transformer, ViT)

1 비전 트랜스포머의 등장 배경

전통적으로 이미지 분류와 객체 탐지 문제는 **합성곱 신경망(CNN)** 기반 구조가 중심이었습니다.

ResNet, Inception, EfficientNet 등의 모델은 공간적인 지역 패턴(예: 에지, 텍스처)을 잘 학습하지만,

전체 이미지의 전역적 관계(long-range dependency)를 포착하는 데 한계가 있습니다.

이러한 한계를 극복하기 위해, 자연어 처리(NLP)에서 큰 성공을 거둔 **Transformer 아키텍처**를

이미지 처리에 확장한 모델이 바로 **Vision Transformer(ViT)**입니다.

즉, ViT는 **"이미지를 문장처럼 처리"**한다는 아이디어에서 출발했습니다.

2 비전 트랜스포머(ViT)의 기본 개념

ViT는 입력 이미지를 CNN처럼 2D로 다루지 않고,

일정 크기의 패치(patch)로 나눈 뒤,

각 패치를 하나의 토큰(token)처럼 **벡터로 임베딩**하여

트랜스포머 인코더에 입력합니다.

즉,

"문장에서 단어를 임베딩하는 것처럼, 이미지에서 패치를 임베딩한다."

이 과정 덕분에 트랜스포머가 이미지의 **전역적 문맥(전체 구조)**를 학습할 수 있게 됩니다.

3 ViT의 핵심 구조 구성요소

(1) 패치 분할 (Patch Embedding)

- 입력 이미지를 일정한 크기(예: 16×16 픽셀)의 작은 패치로 분할합니다.
- 각 패치를 1차원으로 평탄화(flatten)하고, 선형층(Linear Projection)을 통해 고정 차원(dimension)의 벡터로 변환합니다.
- 결과적으로 N개의 패치가 각각 하나의 "단어"처럼 트랜스포머 입력이 됩니다.

(2) 위치 임베딩 (Positional Encoding)

- 트랜스포머는 순서 정보가 없기 때문에, 각 패치의 위치를 인식시키기 위해 **위치 임베딩(positional embedding)** 을 추가합니다.
- 이렇게 하면 모델이 이미지 내 공간적 구조(좌표 관계)를 이해할 수 있습니다.

(3) 클래스 토큰 (Class Token)

- BERT의 [CLS] 토큰처럼, 이미지 전체의 표현을 요약하기 위해 **특별한 학습용 벡터([CLS])** 를 입력 시 맨 앞에 추가합니다.
- 최종적으로 이 [CLS] 벡터를 이용해 **이미지 분류 결과**를 얻습니다.

(4) 멀티-헤드 자기어텐션 (Multi-Head Self-Attention)

- ViT의 핵심 블록으로, 모든 패치 간의 상호 관계를 계산합니다.
- 각 패치가 다른 모든 패치의 정보를 참고하며, 이미지 전체의 전역적 패턴(예: 형태, 대칭, 배치)을 학습합니다.
- 다중 헤드(multiple heads)를 사용하여 서로 다른 관점에서 attention을 계산합니다.

(5) 피드포워드 네트워크 (Feed-Forward Layer)

- Self-Attention 이후, 각 토큰에 대해 개별적으로 적용되는 완전연결층입니다.
- 비선형 변환을 통해 더 풍부한 표현을 학습합니다.

(6) 레이어 정규화 (Layer Normalization) & 잔차 연결 (Residual Connection)

- 안정적 학습과 그라디언트 소실 방지를 위해 각 서브블록에 **잔차 연결(skip connection)** 과 **정규화**가 포함됩니다.

(7) 출력층 (Classification Head)

- [CLS] 토큰을 선형층에 통과시켜 최종적으로 분류 결과(클래스 확률)를 출력합니다.

4 ViT와 CNN의 비교

구분	CNN	Vision Transformer
기본 단위	지역적 필터(커널)	전역적 토큰 간 어텐션

구분	CNN	Vision Transformer
구조	계층적(계단식)	평면적(패치 단위의 병렬 처리)
전역 정보	제한적 (큰 커널 필요)	자연스럽게 전역 컨텍스트 학습
데이터 필요량 적은 편 (데이터 효율적)	매우 많음 (사전학습 필수)	
성능	소규모 데이터에 강함	대규모 데이터에서 우수함

ViT는 대규모 데이터셋(예: ImageNet-21k, JFT-300M)에서 CNN보다 더 높은 정확도를 달성했으며, 전이학습(Transfer Learning) 성능도 매우 뛰어납니다.

5 ViT의 주요 활용 분야

(1) 이미지 분류 (Image Classification)

- 가장 기본적인 응용 분야로, 입력 이미지를 하나의 클래스로 분류합니다.
- 예: 고양이 vs 개, 숫자 인식(FashionMNIST 등)

(2) 객체 탐지 (Object Detection)

- DETR(Detection Transformer)와 결합되어, 이미지 내 물체의 위치와 클래스 정보를 동시에 예측합니다.

(3) 시맨틱 세그멘테이션 (Semantic Segmentation)

- U-Net과 유사한 디코더 구조와 결합해, 각 픽셀 단위로 클래스를 분류하는 데 활용됩니다. (예: DeepLab, Segmenter 등)

(4) 멀티모달 인식 (Text-Image Recognition)

- CLIP, ALIGN 등에서는 텍스트와 이미지를 함께 입력받아 텍스트-이미지 매칭이나 검색에 활용됩니다.

6 파이토치 기반 실습 흐름

10장에서는 ViT의 구조를 직접 구현하고 실험하는 예제가 포함되어 있습니다.

1. 사전학습된 ViT 모델 불러오기

2. `from transformers import ViTForImageClassification`

3. `model = ViTForImageClassification.from_pretrained("google/vit-base-patch16-224")`

4. 데이터 전처리

- transformers의 `AutoImageProcessor`를 이용하여 이미지 리사이즈, 정규화, 텐서화 수행
- `DataLoader`로 배치 구성

5. 학습 설정 (`TrainingArguments`, `Trainer`)

- 학습률, 배치 크기, 평가 전략 등 설정
- 손실 함수: `CrossEntropyLoss`
- 평가 지표: Accuracy, F1-score 등

6. 모델 학습 및 평가

- `trainer.train()` 으로 미세조정(Fine-tuning) 수행
- `trainer.evaluate()`로 테스트 데이터 평가

7. 모델 저장 및 배포

- 학습이 완료된 모델을 `.save_pretrained()`로 저장하고 실제 서비스나 프로젝트에 바로 활용할 수 있도록 안내

7 ViT의 실전 응용과 확장

• 데이터 효율성 향상 모델

→ DeiT (Data-efficient Image Transformer): Knowledge Distillation으로 소규모 데이터에서도 우수한 성능

• 하이브리드 구조

→ CNN + Transformer 결합형 (예: Swin Transformer, ConvNeXt 등)
CNN의 지역 정보 + Transformer의 전역 정보를 모두 활용

• 산업 응용 사례

- 의료영상(병변 탐지, MRI 분석)
- 자율주행(객체 인식, 거리 판단)
- 위성 이미지 분석(지형 분류)
- 텍스트-이미지 검색 서비스(CLIP 기반 추천)

8 학습 및 배포 단계 요약

- **학습 (Training):** 사전학습 모델 불러오기 → 전이학습(Fine-tuning)
- **검증 (Evaluation):** F1, Accuracy 등 지표로 모델 성능 평가
- **배포 (Deployment):** PyTorch 또는 HuggingFace Hub에 업로드하여 API 형태로 서비스화

이 과정 전반을 통해 독자는 **이론적 이해부터 실무적 구현까지 완결된 ViT 학습 경험**을 얻게 됩니다.